

# How to learn, investigate and explain

[Programme home](#) · [Student lessons](#)

## Your route through a lesson

Remember → watch a worked example → try with help → explain on your own → investigate → try a changed case → fix a mistake → return later.

## Remember first

Close your notes. Explain an older idea and give an example. Then check it. An imperfect first answer shows what you need to practise.

## Understand the example

Point to the inputs, the deciding step and the result in the picture. Say what they mean in your own words. Cover the result and predict it before checking.

## Try with less help

Write the given facts and choose a procedure. Try each step. Ask for a hint when you need it, then try the next step yourself. Explain your reasoning to a partner; both people keep their own answer.

## Show what you understand

Try the named Independent case before receiving feedback. Explain your steps, your result and what remains unknown. Write whether you worked alone, used a hint or followed an example. Getting help is useful; it tells your teacher what to practise next.

## Investigate honestly

Write what you expected and what actually happened. If you only traced code on paper, write “not run”. Keep an unexpected result: explaining it is part of learning.

## Use feedback

Find the first step that needs fixing. Change it and explain why. Later, try another case with different facts. Remembering one answer is different from understanding the idea.

## Keep your project evidence

Save your inputs, traces, tests and revisions. Include results that did not work and explain the limits. Print or save your workbook to retain typed answers. Hosted drafts and progress stay in the current browser; print or download evidence for your portfolio. Clear drafts on shared devices.

## Take the time you need

A week can take several classes. Complete the connected topics as well as the main example. Your teacher checks your readiness before new coding tasks.

## Your academy end goal

Learn to explain how AI works, build a small system, test it against a simple starting method, investigate mistakes and defend your conclusions with evidence. Your project should show what you tried, what changed and what remains uncertain.